

## Bug 1: SQL Operator Precedence

### 1. Location:

'TaskRepository.java' L14-17 (backend / SQL layer);  
also 'search\_tasks.sql' L10-13 and  
'task-search-package.sql' L52-55, L66-69

### 2. Discovery:

Read the native SQL query and traced operator precedence - AND binds before OR in SQL, just like in most programming languages. Confirmed by checking that archived tasks (IDs 20, 21) would appear in unfiltered searches.

### 3. Root Cause:

The WHERE clause

'archived=FALSE AND title LIKE:term OR description LIKE:term AND status=:status' parses as '(archived AND status)' due to missing parentheses. This means any title match bypasses both the archive filter and status filter.

### 4. Fix & Reasoning:

Added parentheses: 'AND (title LIKE:term OR description LIKE:term)'. This is the minimal change - one pair of parens - that correctly groups the OR without restructuring the query.

Alternative: rewrite as two UNION query, but that's unnecessary complexity for a AND/OR grouping issue.

## Bug 2: Thread.sleep() in Controller

1. Location:  
'TaskController.java' L36-42 (backend layer)
2. Discovery:  
Noticed API responses were sluggish during manual testing. Traced to an explicit Thread.sleep() call that delays responses by up to 1000ms for short/empty queries.
3. Root Cause:  
The code calculates 'complexity score = 10 \* query.length()' and sleeps for 'complexity score \* 100' milliseconds. An empty search (the default page load) sleeps for a full second, blocking a Tomcat thread pool thread.
4. Fix & Reasoning:  
Removed the Thread.sleep block entirely. The complexity score log line was kept since it's harmless diagnostic info.

### Alternative:

Reduce the sleep to a small value - but sleeping on a request thread is never appropriate in a synchronous servlet architecture. If throttling were needed, use async processing or a rate limiter.

## Bug 3: Loading state Not Cleared on Error

### 1. Location:

'useTasks.js' L19-21 (Frontend layer)

### 2. Discovery:

Reasoned through the state machine: `setLoading(true)` is called at the start of `useEffect`, `setLoading(false)` is called in `.then()` but NOT in `.catch()`. If the backend is down, the UI would be stuck.

### 3. Root Cause:

The `.catch()` handler sets the error message but never calls `setLoading(false)`. Since `TaskTable` checks 'if(loading)' before 'if(error)', the loading state masks the error state permanently.

### 4. Fix & Reasoning:

Added '`setLoading(false)`' inside the `.catch()` block. This is the standard pattern - loading should be cleared in both success and error paths.

### Alternative:

Use a `.finally()` block, but that would require restructuring the promise chain and `.then()/.catch()` is already the established pattern here.

## Bug 4: Page Not Reset on Filter Change

### 1. Location:

'App.jsx' L24-25 (Frontend layer)

### 2. Discovery:

Tested the UI flow: navigate to page 3, then type a search term that returns fewer than 30 results. The page stays at 3, showing "No tasks found" with no way to navigate back (pagination controls disappear when total Pages < 2).

### 3. Root cause:

The SearchBar and StatusFilter components were wired directly to 'setQuery' and 'setStatus', which update their respective state but leave 'page' unchanged. The 'useTasks' hook then fetches with the old page number against a smaller result set.

### 4. Fix & Reasoning:

Created 'handleQueryChange' and 'handleStatusChange' wrapper functions that call both the setter and 'setPage(1)'. This keeps the fix local to App.jsx without modifying child components.

### Alternative:

Reset page inside useEffect in useTasks.js, but that would create a double fetch (one with old page, one with reset page) and violate the principle that the parent should own the state it passes down.